

Assignment 4

Colton Brewer

Exercise 1

1.)

Hyndman and Fan introduced 9 different definitions of sample quantiles because the quantile can fall between two observations. Each of these

1. Let N be the count of observations in the ordered sample, and q be the desired quantile. q is rounded up to the nearest x/N for integer x . the quantile takes the x th value in the ordered sample.
2. Same as (1.), but in the boundary case where there exists integer x for $q = x/N$, the quantile takes the mean of the x th and $(x+1)$ th value in the ordered sample.
3. Same as (1.), but instead of rounding up to the nearest x/N for integer x , it rounds tot the closest x/N (which may be less than q). For situations where $q - x/N = (x+1)/n - q$, the quantile takes the x th value on the ordered sample when x is even, otherwise it takes the $(x+1)$ th value.
4. Let N be the count of observations in the ordered sample, and q be the desired quantile. if $q = x/N$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists integer x such that $x/N < q < (x+1)/N$. let s_x and $s_{(x+1)}$ represent the x th and $(x+1)$ th value in the ordered sample. The quantile takes the value $(qN-x)(s_{(x+1)}-s_x)+s_x$.
5. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $(x-0.5)/N$. If $q = (x-0.5)/N$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer x such that $(x-0.5)/N < q < (x+0.5)/N$. Let s_x and $s_{(x+1)}$ represent the x th and $(x+1)$ th values in the ordered sample. The quantile takes the value $(qN-x+0.5)(s_{(x+1)}-s_x)+s_x$.
6. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $x/(N+1)$. If $q = x/(N+1)$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer such that $x/(N+1) < q < (x+1)/(N+1)$. The quantile takes the value $(q(N+1)-x)(s_{(x+1)}-s_x)+s_x$.

7. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $(x-1)/(N-1)$. If $q = (x-1)/(N-1)$ for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer x such that $(x-1)/(N-1) < q < x/(N-1)$. The quantile takes the value $(q(N-1)-x_{-1})(s_{(x+1)}-s_x)$.
8. Let N be the count of observations in the ordered sample, and q be the desired quantile. the x th value in the ordered sample is assigned the position $(x-1/3)/(N+1/3)$. If q equals this position for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer x such that $(x-1/3)/(N+1/3) < q < (x+2/3)/(N+1/3)$.
9. Let N be the count of observations in the ordered sample, and q be the desired quantile. The x th value in the ordered sample is assigned the position $(x-3/8)/(N+1/4)$. If q equals this position for some integer x , the quantile takes the x th value in the ordered sample. Otherwise, there exists an integer such that $(x-3/8)/(N+1/4) < q < (x+5/8)/(N+1/4)$. The quantile takes the value $(q(N+1/4)-x+3/8)(s_{(x+1)}-s_x)+s_x$.

For 5, 6, 8 and 9, values of q close to 0 or 1 can fall outside the positions assigned to the observations. In these cases, the quantile takes the value of the sample minimum or maximum. Note that the differences between these methods grow smaller as sample size decreases. While R uses type 7, Hyndman and Fan recommends type 8 as it is an approximately median unbiased estimator that does not depend on a particular underlying distribution.

2.)

A biased estimator is a formula where the expected value does not equal the true population parameter.

```
# Create function for lambda using the interquartile range (type 8) method
lambda_iqr <- function(x) {
  sample_iqr <- IQR(
    x,
    type = 8
  )

  return(log(3)/sample_iqr)
}

# Create function for bootstrap sample
lambda_iqr_corrected <- function(x, size = 200) {
  lambda_iqr_initial <- lambda_iqr(x)
  lambda_bootstrap <- replicate(size, {
    bootstrap_sample <- rexp(
      n = length(x),
```

```

    rate = lambda_iqr_initial
  )
  lambda_iqr(bootstrap_sample)
})

# Estimate bias
estimated_bias <- mean(lambda_bootstrap) - lambda_iqr_initial

# Create the first order bias corrected estimate
lambda_corrected <- lambda_iqr_initial - estimated_bias

return(lambda_corrected)
}

```

3.)

```

# Create the ml estimator function
lambda_ml <- function(x) {
  1/mean(x)
}

# Run the Monte Carlo experiment
run_simulation <- function(lambda, n, M = 5000, size = 200) {
  estimates <- replicate(M, {

    x <- rexp(
      n = n,
      rate = lambda
    )

    c(
      ML = lambda_ml(x),
      IQR = lambda_iqr(x),
      Bootstrap = lambda_iqr_corrected(x, size = size)
    )
  })

  estimates <- as.data.frame(t(estimates))

  tibble(

```

```

    lambda = lambda,
    n = n,
    estimator = c("ML", "IQR", "Bootstrap"),
    MSE = c(
      mean((estimates$ML - lambda)^2),
      mean((estimates$IQR - lambda)^2),
      mean((estimates$Bootstrap - lambda)^2)
    )
  )
}

# Run every combination
parameter_space <- expand_grid(
  lambda = c(0.1, 1, 10),
  n = c(5, 30, 100)
)

monte_carlo_results <- parameter_space |>
  pmap_dfr(
    \(lambda, n) {
      run_simulation(
        lambda = lambda,
        n = n,
        M = 5000,
        size = 200
      )
    }
  )

```

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$ML - lambda: longer object length is not a multiple of shorter object length

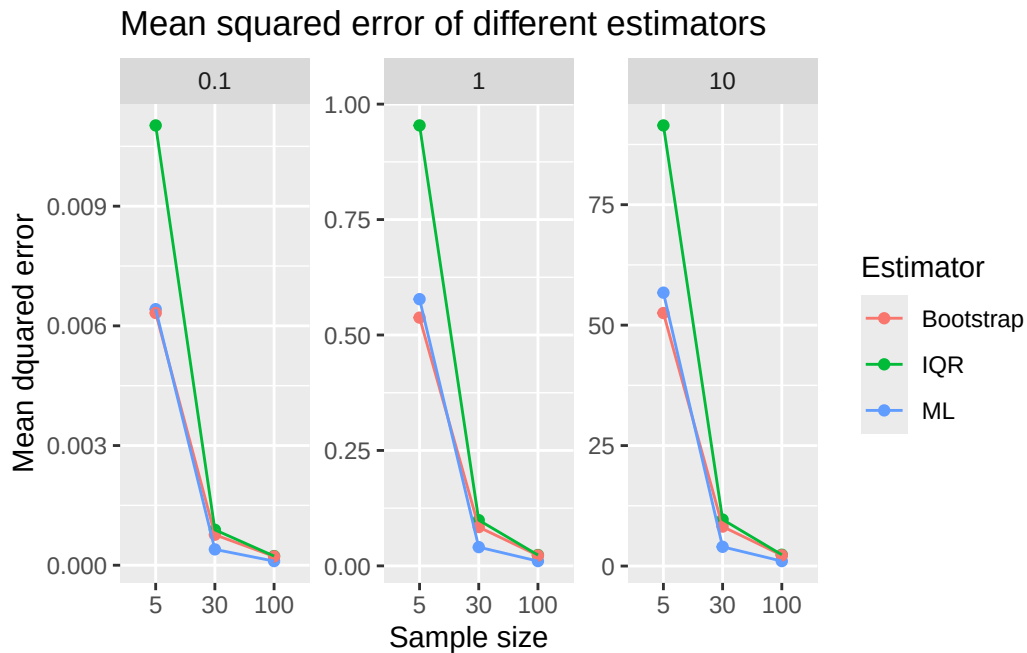
Warning in estimates\$IQR - lambda: longer object length is not a multiple of shorter object length

Warning in estimates\$Bootstrap - lambda: longer object length is not a multiple of shorter object length

```
# Show the data in a wide format
(monte_carlo_results_wide <- monte_carlo_results |>
  pivot_wider(
    names_from = estimator,
    values_from = MSE
  ) |>
  pander())
```

lambda	n	ML	IQR	Bootstrap
0.1	5	0.006419	0.01102	0.006324
0.1	30	0.0003969	0.0008878	0.000764
0.1	100	0.0001029	0.0002298	0.0002208
1	5	0.5777	0.9539	0.5377
1	30	0.04063	0.09908	0.08405
1	100	0.01036	0.02353	0.02267
10	5	56.74	91.45	52.52
10	30	3.99	9.65	8.189
10	100	1.022	2.343	2.239

```
# Plot the results
monte_carlo_results |>
  ggplot(
    aes(
      x = factor(n),
      y = MSE,
      colour = estimator,
      group = estimator
    )
  ) +
  geom_point() +
  geom_line() +
  facet_wrap(
    ~ lambda,
    scales = "free_y"
  ) +
  labs(
    title = "Mean squared error of different estimators",
    x = "Sample size",
    y = "Mean squared error",
    colour = "Estimator"
  )
```



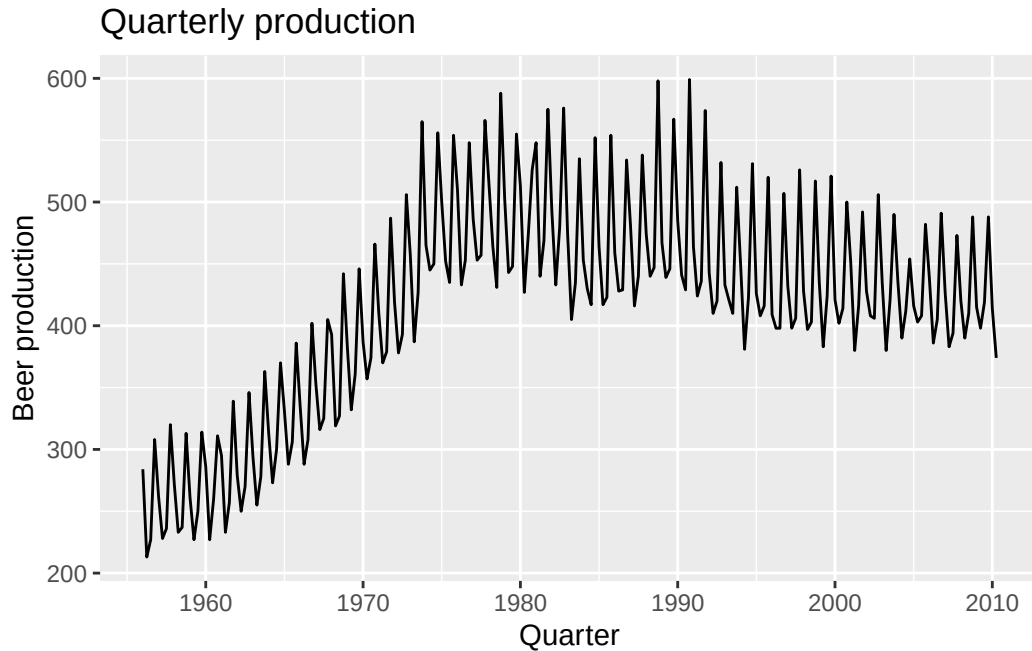
Why 5000 replications was chosen for each parameter combinations

Exercise 2

1.)

The frequency of the ausbeer time series is 4 observations in each year, meaning that it's quarterly data. It runs from Q1 1956 to Q2 2010, which is confirmed in the package documentation. The frequency can also be seen in the time series plot below, as there is a clear seasonal pattern occurring every 4 instances.

```
# Time series line graph
autoplot(ausbeer) +
  labs(
    title = "Quarterly production",
    x = "Quarter",
    y = "Beer production"
  )
```



2.)

The series can be seen having a strong linear increasing trend with seasonal pattern until 1974. Beyond that, an ever so slightly decreasing trend follows, with more annual variations than before 1974. The long term trend is present, but it isn't consistent through this historical period, being an increase before a smaller decline starting around 1975.

3.)

Seasonality is present through this time series, where the largest trough in a period is in quarter 2, before increasing the next two subsequent quarters to reach the largest peak in quarter 4, before decreasing in quarter 1. This pattern is followed in most years, but there are instances where Q3 has the largest trough.

4.)

The time series displays heteroscedasticity meaning that the variability isn't constant over time. In the earlier years when there was a relatively constant increase (accounting for seasonality) the variability remains steadfast. Once the series transitions away from the steady increase over time, the variability also increases. Near the end of this series, it seems to slowly decrease. A log transformation may help stabilize this irregular variance to some degree, but

this method has best results when variability scales with the observed outcome variable (beer production), which isn't the case here since variation was consistent in the earlier increasing years.

5.)

Unusual observations:

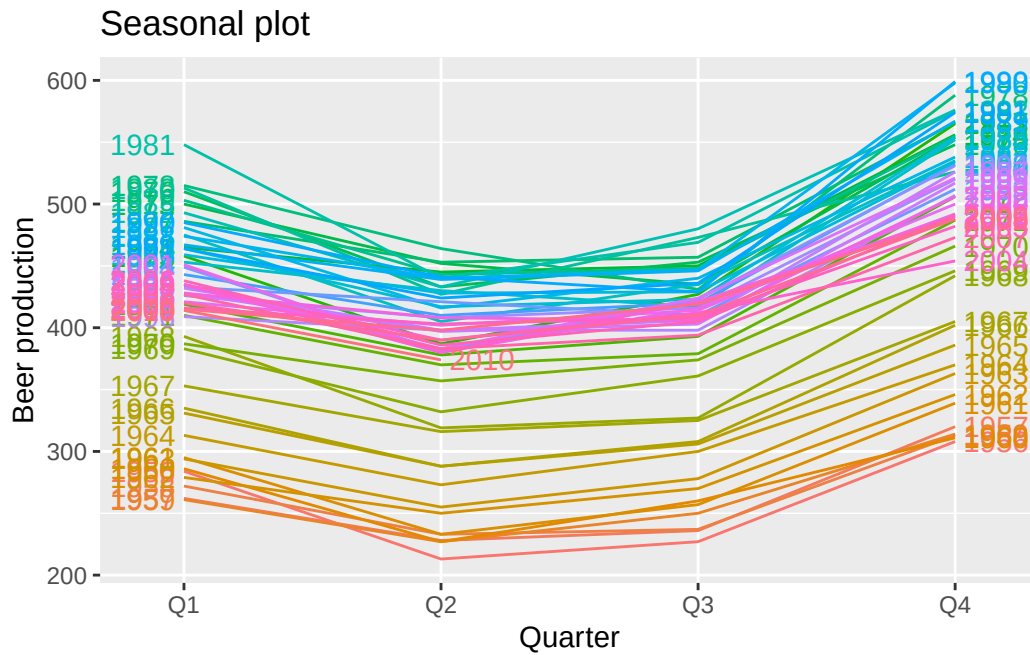
large jump before the trend shift in the trough

A few high peaks around 1990

Low troughs in roughly 1984 and 1994

6.)

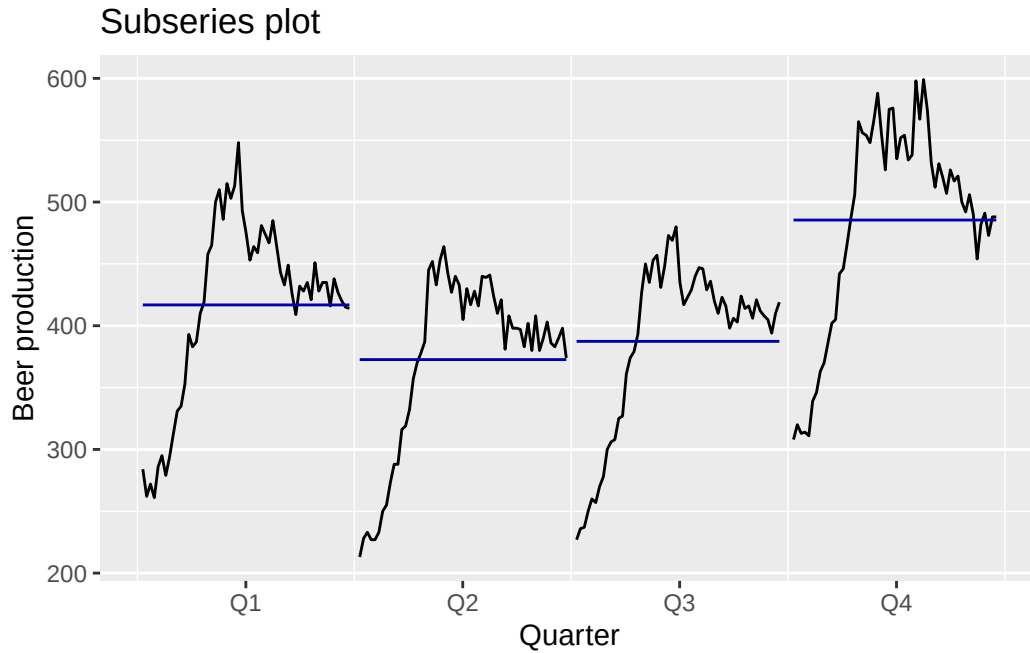
```
ausbeer |> ggseasonplot(  
  year.labels = TRUE,  
  year.labels.left = TRUE  
) +  
  labs(  
    title = "Seasonal plot",  
    x = "Quarter",  
    y = "Beer production"  
  )
```



There's a strong pattern of Q4 being the highest and Q1 being the second highest in terms of beer production. Q2 and Q3 are relatively interchangeable.

7.)

```
ausbeer |> ggsubseriesplot() +
  labs(
    title = "Subseries plot",
    x = "Quarter",
    y = "Beer production"
  )
```



This plot shows that the ranking of quarters from highest beer production to lowest is 4, 1, 3, then 2. Seasonal pattern isn't stable over time, as there is a higher variance shown after the initial, stable increase. If it were stable over this period, there would be much less variance.